

FSI-Mythos on AgentCore

국내 금융사를 위한 Claude 기반 자율 보안 스캐닝 플랫폼. Anthropic Mythos의 8-Phase 스캐폴드를 Strands Agents SDK와 Amazon Bedrock AgentCore로 재구현하고, Cognito·CloudFront·WAF·DynamoDB를 갖춘 풀스택으로 AWS에 배포·검증한 시스템의 전체 설계 문서.

2026-06-02

AWS us-west-2

Claude Opus 4.8/4.7/4.6

AppSec 표준

SECTION 01

개요

FSI-Mythos는 소스코드에서 보안 취약점을 **자율적으로 발견·검증**하는 멀티에이전트 시스템입니다. 단순 패턴 매칭 (SAST)을 넘어, Claude의 의미론적 추론으로 컴포넌트 간 상호작용에서 발생하는 논리적 결함까지 탐지하는 것을 목표로 합니다.

핵심 설계 원칙

- 적대적 검증** — Hunter가 발견한 취약점을 Challenger가 반박하고 Validator가 최종 판정해 오탐(FP)을 줄임
- 방어 목적 한정** — `scan_scope="defensive"`, 익스플로잇 무기화 단계 미구현
- 역할별 모델 분배** — 추론 난이도·비용에 따라 Opus 4.8/4.7/4.6 차등 배치
- AppSec 표준 준수** — 인증·암호화·최소권한·WAF를 기본 적용
- 재현성** — pass@k 독립 실행으로 발견의 일관성 확보

8

Phase 파이프라인

4

에이전트 역할

3

Opus 모델 티어

4

CDK 스택

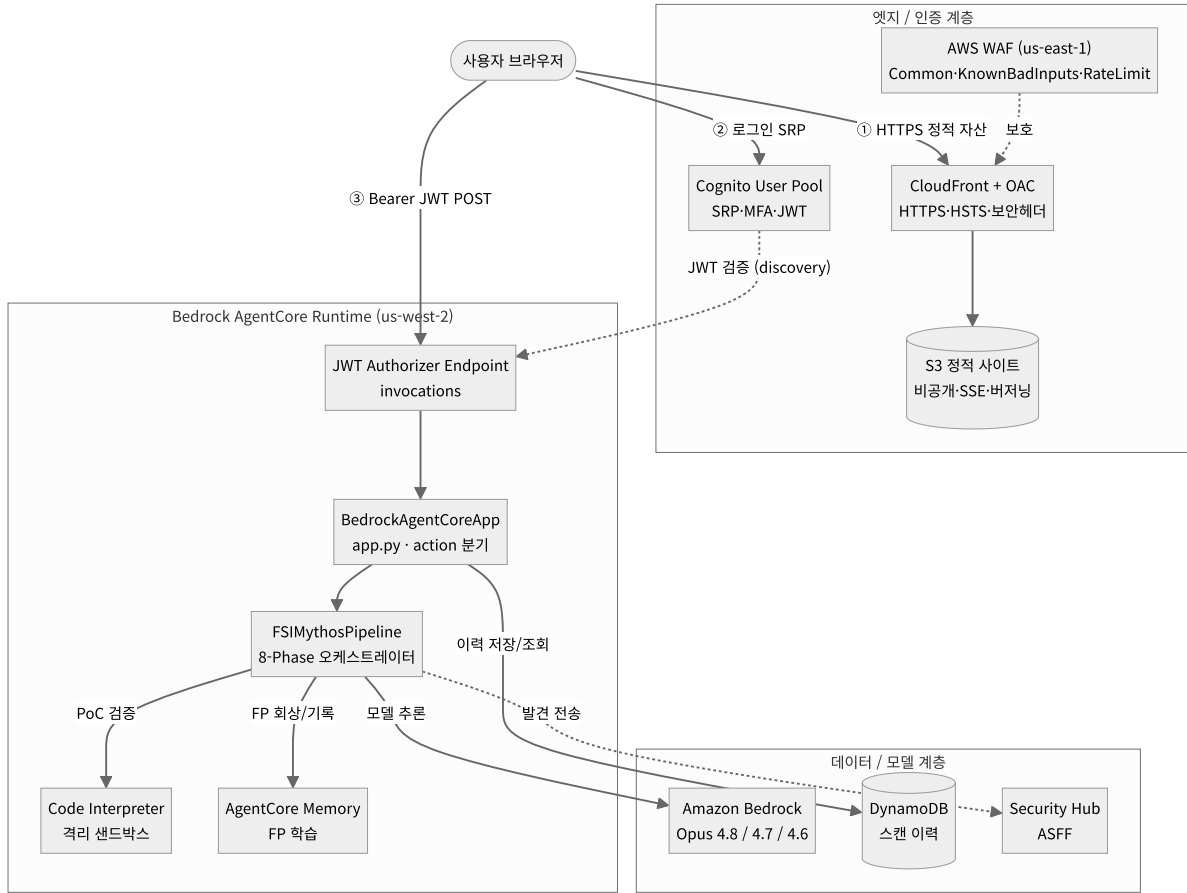
⚠️ 면책

본 문서의 "Mythos"는 분석 대상 개념이며, 시스템은 공개 Claude 모델로 방어적 취약점 발견만 수행합니다. 실제 익스플로잇 생성·공격에는 사용할 수 없습니다.

SECTION 02

시스템 아키텍처

3계층 구조: **엣지/인증 계층**(CloudFront·WAF·Cognito), **실행 계층**(AgentCore Runtime), **데이터/모델 계층**(Bedrock·DynamoDB).

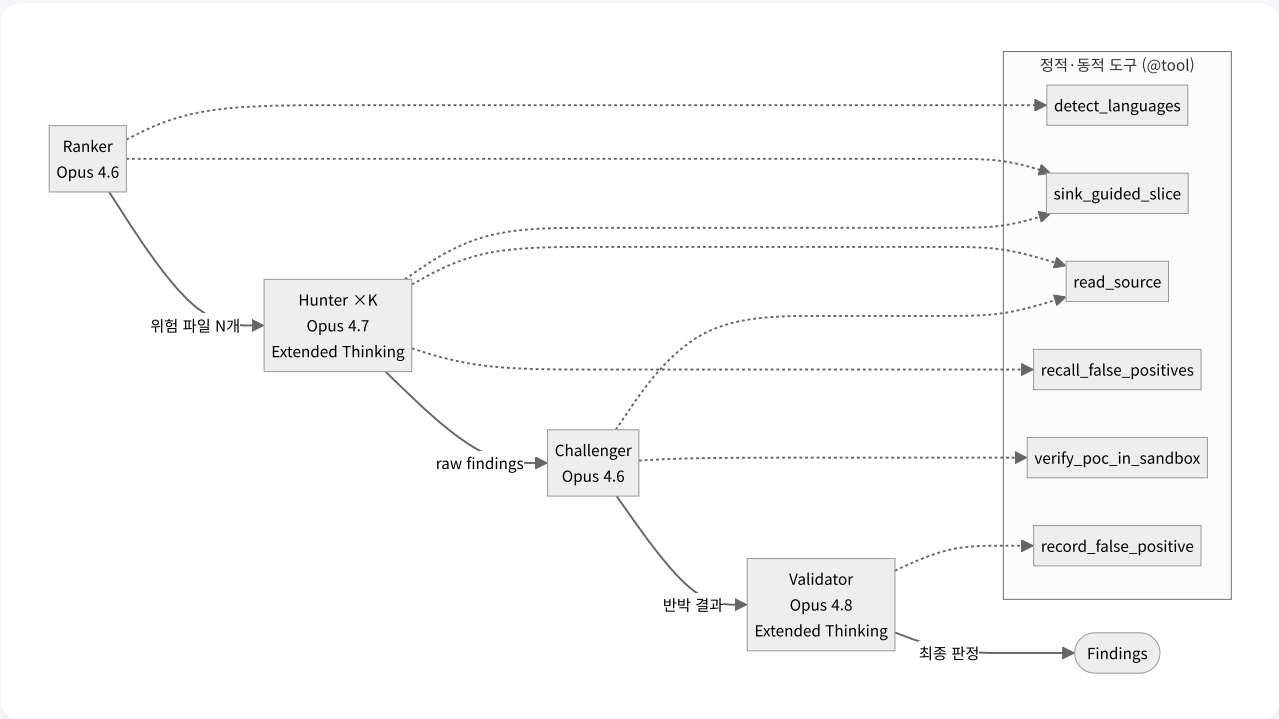


구성요소 책임

계층	구성요소	책임
엣지/인증	CloudFront + OAC	정적 UI 배포, HTTPS 강제, 보안 응답 헤더, S3 비공개 접근
	WAF	관리형 룰·IP 속도 제한으로 악성 트래픽 차단
	S3	React 빌드 산출물 호스팅 (퍼블릭 전면 차단)
	Cognito	SRP 인증, JWT 발급, AgentCore authorizer 신뢰 기반
실행	AgentCore Runtime	서버리스 컨테이너, JWT 인가, 세션 격리 microVM
	Pipeline	8-Phase 멀티에이전트 오케스트레이션
	Code Interpreter	PoC 크래시 재현 (네트워크 차단 샌드박스)
데이터/모델	Bedrock	Opus 3종 추론 (역할별)
	DynamoDB	사용자별 스캔 이력 영속화
	Security Hub	ASFF 포맷 발견 통합 (선택)

멀티에이전트 스캐폴드

Strands Agents SDK로 4개 역할 에이전트를 구성합니다. 각 에이전트는 **모델 + 시스템 프롬프트 + 도구**의 조합이며, 역할별로 다른 Opus 모델과 Extended Thinking 설정을 사용합니다.



에이전트	모델	Thinking	도구	역할
Ranker	Opus 4.6	off	detect_languages, sink_guided_slice	파일 위험도 랭킹
Hunter	Opus 4.7	adaptive-high	read_source, sink_guided_slice, recall_false_positives	취약점 발견 (K회 독립)
Challenger	Opus 4.6	off	read_source, verify_poc_in_sandbox	적대적 반박·PoC 검증
Validator	Opus 4.8	adaptive-high	record_false_positive	최종 판정·FP 기록

☑ 모델 분배 근거

발견(Hunter)은 다양성·속도 균형의 4.7, 최종 판정(Validator)은 최고 추론력의 4.8, 비용 효율이 중요한 랭킹·반박 (Ranker·Challenger)은 4.6. Challenger의 Thinking을 끈 이유는 4.6+adaptive thinking 스트리밍의 간헐적 ValidationException 회피와 비용·지연 절감 (반박 품질 영향 미미).

SECTION 04

8-Phase 파이프라인

Phase별 상세

Phase	입력 → 출력	핵심 로직
0 언어 감지	경로 → {언어: [파일]}	확장자 매핑, .git/node_modules 등 제외
1 싱크 슬라이싱	파일 → 싱크 컨텍스트	위험 함수(strcpy/system/eval) 전후 20줄 추출
2 파일 랭킹	싱크 밀도 → 상위 N개	금융 특화 기준(인증·암호·거래·노출). 싱크 0이면 비용 절약 폴백
3 힌트	파일 → raw findings	K회 독립 실행(temperature=1.0), 의미론 추론, JSON 보고
3.5 자기도전	findings → 반박	컴파일러 최적화·ASLR·경로 도달성 검토, 샌드박스 PoC. 예외 격리
4 검증	반박 → 최종 판정	CONFIRMED/LIKELY/DISMISSED/ESCALATE, 신뢰도 0~1
6 집계	판정 → 보고서	심각도 집계, ASFF 변환, CI/CD 게이트
7 FP 메모리	DISMISSED → 학습	오탐 패턴을 AgentCore Memory에 영속화

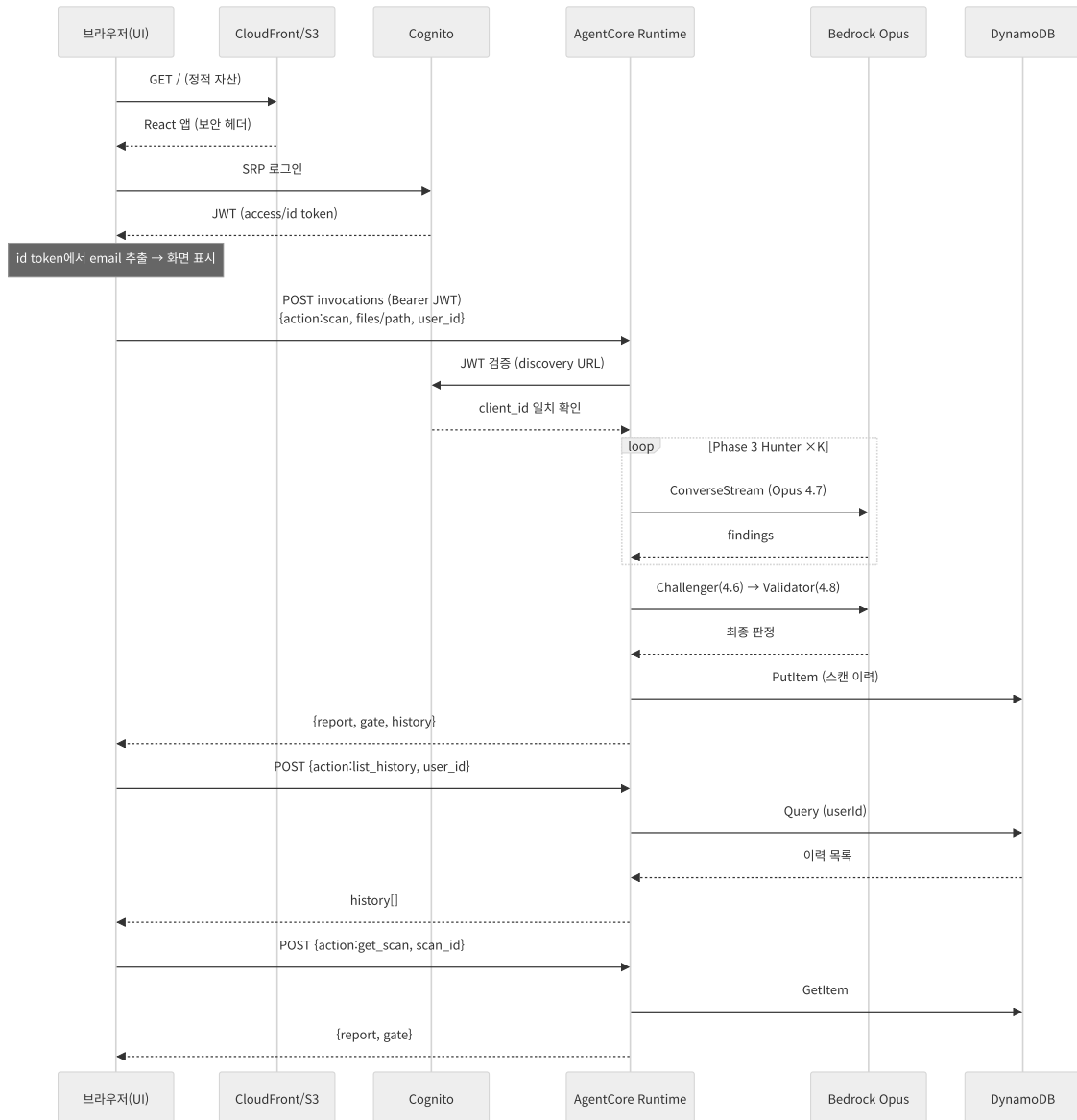
☒ pass@k 재현성

Phase 3에서 Hunter를 K회 독립 실행하고, 동일 (파일·제목) 발견의 등장 빈도를 신뢰도에 반영합니다. 여러 번 발견될수록 진성 취약점일 확률이 높습니다.

SECTION 05

요청 시퀀스

브라우저에서 로그인 후 스캔을 실행하고 이력을 조회하는 전체 흐름.



SECTION 06

데이터 모델 · 이력 관리

Finding 데이터 모델

```

@dataclass
class Finding:
    id: str          # fsi-{sha1[:16]}
    title: str
    file_path: str
    line_range: tuple
    severity: Severity # critical | high | medium | low | info
    cwe_id: Optional[str] # CWE-XXX
    description: str
    exploitation_scenario: str
    patch_suggestion: str
    confidence: float   # 0~1 (검증 후)
  
```

chain_potential: bool # 체이닝 가능성
verdict: Verdict # confirmed | likely | dismissed | escalate
validated: bool

DynamoDB 스캔 이력 스키마

SCAN_HISTORY			
string	userId	PK	Cognito email - partition key
string	scanId		createdAt and uuid - sort key
string	createdAt		ISO8601
string	projectPath		scan target
number	maxFiles		config
number	passAtK		config
string	summary		JSON summary
string	report		JSON full report
string	gate		JSON CI-CD gate

속성

설명

userId (PK)

Cognito 이메일 — 사용자별 데이터 격리

scanId (SK)

createdAt#uuid8 — ScanIndexForward=false로 최신순 조회

summary/report/gate

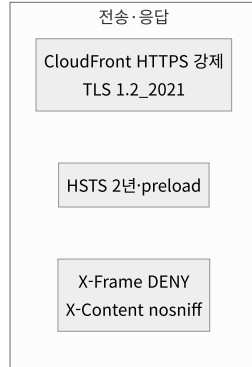
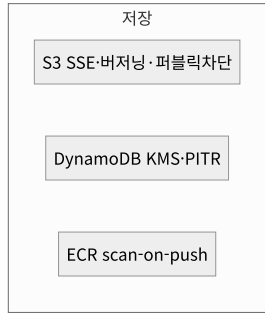
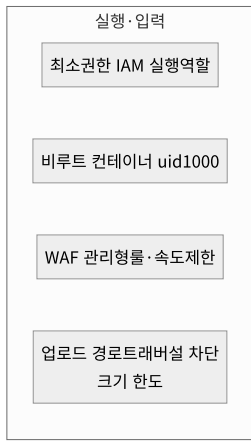
float 직접 저장 불가 → JSON 문자열로 직렬화

☑ 이력 액션 분기

scan (실행+자동저장) / **list_history** (목록) / **get_scan** (상세). 스캔 완료 시 결과를 자동 저장하며, 저장 실패해도 결과 반환은 유지(격리).

SECTION 07

보안 설계 (AppSec 표준)

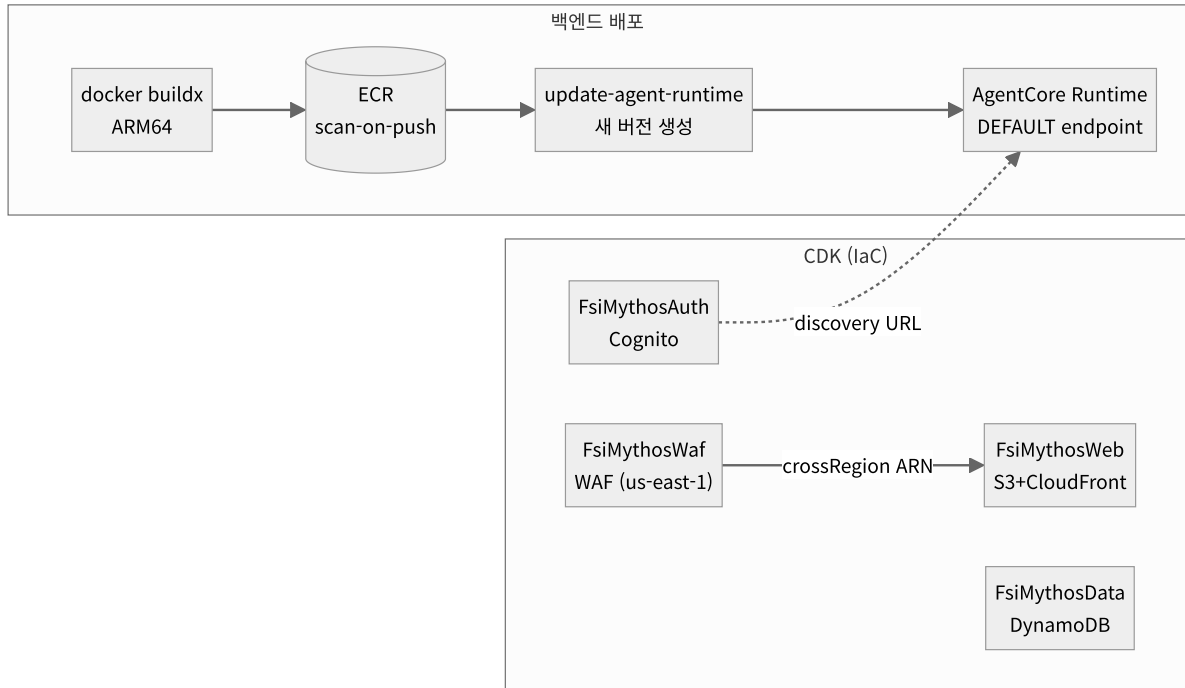
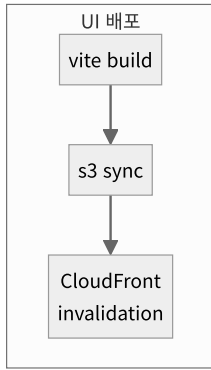


위협	완화책
무단 접근	Cognito SRP + AgentCore JWT authorizer (access token client_id 검증)
중간자 공격	HTTPS 강제, TLS 1.2_2021, HSTS preload
클릭재킹/MIME 스니핑	X-Frame-Options DENY, X-Content-Type-Options nosniff
데이터 유출	S3 퍼블릭 차단·OAC, DynamoDB 암호화, 사용자별 파티션 격리
경로 트래버설(업로드)	normpath 정규화 후 base 디렉토리 탈출 차단 (검증됨)
DoS(업로드)	파일 200개·파일당 1MB·전체 20MB 한도
권한 상승	실행역할 최소권한 (Opus 3종 + Code Interpreter + DynamoDB R/W + 로그만)
봇·악성 트래픽	WAF Common·KnownBadInputs + IP 속도 제한 2000/5분

SECTION 08

배포 아키텍처

인프라는 **CDK 4스택**(IaC), 백엔드는 **AgentCore CLI + ARM64 컨테이너**로 분리 배포합니다.



⚠ 배포 핵심 주의사항

- ① WAF는 CloudFront용이라 **us-east-1** 에 생성 (crossRegionReferences로 참조). ② AgentCore는 이미지 푸시만으로 갱신 안 됨 — **update-agent-runtime** 으로 새 버전 생성해야 DEFAULT 엔드포인트가 최신 이미지 사용. ③ 모델 리전은 컨테이너 **AWS_REGION** 신뢰 (payload region 무시).

SECTION 09

성능 평가 · 벤치마크

AI 기반 취약점 탐지 시스템의 성능은 **공개 벤치마크 데이터셋**에서 표준 지표로 측정하고, 동일 ground truth에서 기존 SAST 도구와 비교합니다.

9.1 벤치마크 데이터셋

벤치마크	유형	규모	언어	특징
NIST SARD / Juliet	합성	45만+ 케이스, 150+ CWE	C/C++/Java/PHP/C#	명확한 ground truth, TP/FP/FN/TN 정밀 측정

OWASP Benchmark	실행형 웹앱	Java 2,740 / Py 1,230	Java/Python	SAST 표준 비교, Youden Index 점수
PrimeVul	실세계 함수	~7k 취약 / ~229k 정상	C/C++	라벨 정제·중복제거, 현실적 평가 권장
DiverseVul	실세계 함수	18,945 / 330,492	C/C++	295+ 프로젝트, 150 CWE
CVEfixes	CVE-수정커밋	11,873 CVE / 138,974 함수	다중	실세계 ground truth 구축용, 272 CWE
CyberSecEval (Meta v4)	LLM 보안	다중 태스크	다중	안전 코드 생성·프롬프트 인젝션
CVE-Bench	실세계 익스플로잇	critical CVE	웹	실제 악용 가능성(성공률 ≤13%)
SWE-bench Verified	이슈 해결	500 검증	Python	코드 수정·패치 능력
CyBench	CTF 에이전트	40 tasks	-	에이전트 자율 공격 평가

⚠ 데이터 누출 함정

PrimeVul 연구: BigVul에서 F1 68%이던 모델이 라벨 정제·중복 제거 후 F1 3%로 급락

. 기존 벤치마크가 성능을 크게 과대평가함. 반드시

시간순 분할(chronological split)

과

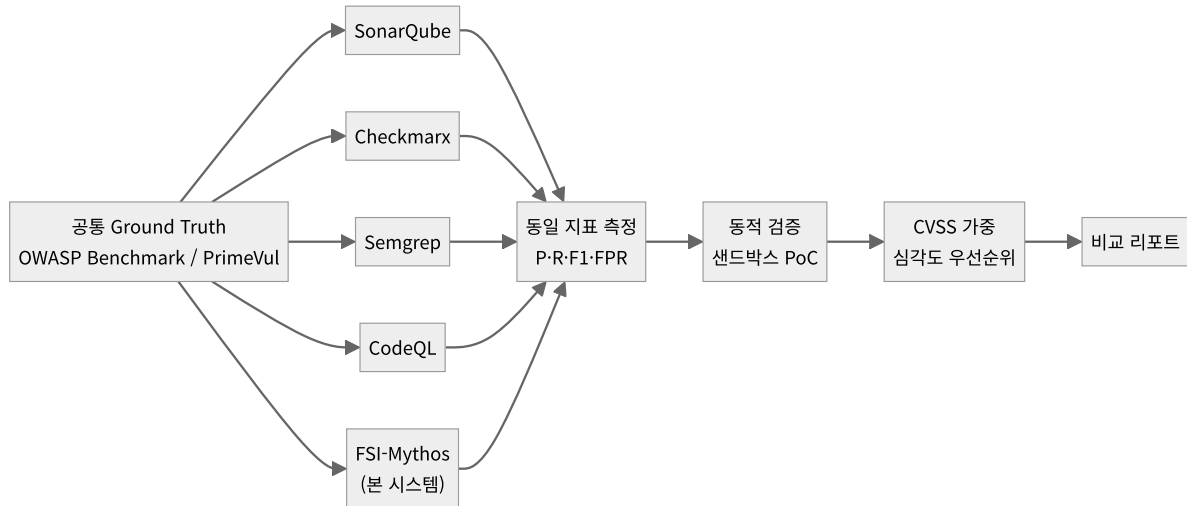
중복 제거

로 통제할 것.

9.2 평가 지표

지표	공식	의미·금융권 함의
Precision	$TP/(TP+FP)$	오탐 통제 — 보안팀 피로도·신뢰성 직결
Recall (TPR)	$TP/(TP+FN)$	미탐 통제 — 놓친 취약점 = 사고 위험
F1	$2PR/(P+R)$	균형 지표 (PrimeVul·BigVul 주 지표)
FPR	$FP/(FP+TN)$	오탐률
Youden Index	(민감도+특이도)-1	OWASP Benchmark 점수 (추측선 대비)
pass@k	k회 중 1회+ 성공률	재현성 — 본 시스템 pass_at_k 와 직결
CWE 분류 정확도	예측/실제 CWE 일치율	유형 식별 능력
익스플로잇 성공률	PoC 성공 비율	실제 악용성 — Challenger 샌드박스 검증과 연계

9.3 SAST 도구 비교 방법론



1. **공통 ground truth** — OWASP Benchmark/PrimeVul에서 SAST 도구와 본 시스템을 동일 데이터로 TP/FP/FN/TN 측정
2. **합성 vs 실세계 통제** — SARD(명확·단순) + 실세계 데이터셋(현실적·노이즈) 병행
3. **동적 검증** — 정적 라벨 매칭을 넘어 Challenger 샌드박스 PoC로 실제 악용성 확인
4. **CVSS 가중** — 심각도(Critical 9.0~10.0)로 발견 우선순위 평가

9.4 본 시스템 내장 평가 메커니즘

메커니즘	구현 위치	효과
pass@k 재현성	Phase 3 Hunter K회 독립 실행	발견 일관성, 빈도→신뢰도 반영
적대적 검증	Phase 3.5 Challenger + 4 Validator	오탐(FP) 자동 제거
FP 메모리	Phase 7 AgentCore Memory	오탐 패턴 학습, 반복 방지
신뢰도 점수	Finding.confidence 0~1	CI/CD 게이트 임계값 활용
샌드박스 검증	verify_poc_in_sandbox	크래시 재현으로 실제 악용성 확인

☑ 금융권 평가 권장안

① OWASP Benchmark로 SAST 대비 정량 비교(P/R/F1) → ② PrimeVul로 실세계 현실성 검증 → ③ 자사 레거시 C/C++ 코드의 과거 CVE 패치를 ground truth로 회귀 평가 → ④ Precision 우선(오탐 최소화)으로 보안팀 신뢰 확보 후 Recall 확대.

SECTION 10

설계 결정 · 교훈

이슈	원인	해결
----	----	----

thinking 미지원 에러	Opus 4.7/4.8은 구 <code>thinking.type=enabled</code> 미지 원	<code>thinking.type=adaptive</code> + <code>output_config.effort</code> 사용
이미지 갱신 안 됨	Runtime이 버전에 이미지 다이제스트 고정	<code>update-agent-runtime</code> 으로 새 버전 생성
"model identifier invalid"	UI가 보낸 <code>region=ap-northeast-2</code> 를 모델 리전으 로 사용 → us.* 프로필 미가용	컨테이너 <code>AWS_REGION</code> 신뢰, payload region 무시
JWT 401	ID token(aud) 사용	access token의 <code>client_id</code> 클레임 매칭
4.6 간헐 실패	4.6+adaptive thinking 스트리밍 불안정	Challenger thinking off + Phase 3.5 예외 격 리
로컬 폴더 접근	AgentCore는 격리 컨테이너	File System Access API로 파일 읽어 업로드 → 임시 디렉토리 materialize
우측 상단 UUID 표시	email alias Pool은 Username이 UUID	id token의 email 클레임 표시